Writing CGI and PHP Scripts in Icon and Unicon

Clinton Jeffery and Jonathan Vallejo

Unicon Technical Report: 4a

2011-04-02

Abstract

CGI scripts are programs that read and write information in order to process input forms and generate dynamic content for the world-wide web. This report describes a library that simplifies writing of CGI scripts in Icon.

Unicon Project
http://unicon.org
University of Idaho
Department of Computer Science
Moscow, ID, 83844, USA

1 Introduction

The Common Gateway Interface, or CGI, is a definition of the means by which web servers interact with external programs that assist in processing web input and output. CGI scripts are programs that are invoked by a World-Wide Web server in order to input data from a user, or provide users with pages of dynamically generated content, as opposed to static content found in .html files. The standard reference on CGI are available on the Web at http://hoohoo.ncsa.uiuc.edu/cgi/ Icon is an ideal language for writing CGI scripts, because of its strong text processing capabilities. This technical report describes `cgi.icn`, a library of Icon procedures for writing CGI scripts for the World-Wide Web. The library was written by Joe van Meter and Clinton Jeffery. This is volunteer-ware; we welcome additions and corrections, sent to jeffery@cs.uidaho.edu.

`cgi.icn` consists of a number of procedures to simplify CGI input processing and especially the generation of HTML-tagged output from various Icon data structures. To link the library into your program, compile it (`icont -c cgi`) and place the statement

```
link cgi
```

at the top of your Icon program prior to compiling it.

1.0.1 Organization of a CGI Script

CGI scripts are very simple. They process input data supplied by the web browser that invoked the script (if any), and then write a new web page, in HTML, to their standard output. When using cgi.icn the input processing phase is automatically completed before control is passed to your program, which is organized around the HTML code that you generate in response to the user. In fact, cgi.icn includes a `main()` procedure that processes the input and writes HTML header and tail information around your program's output. For this reason, when you use cgi.icn, you must call your main procedure `cgimain()`.

1.0.2 Processing Input

The HTTP protocol includes two means of invoking a CGI script, with different ways of supplying user input, either from the standard input or from a `QUERY_STRING` environment variable. In either case, the input is organized as a set of fields that were given names in the HTML code from which the CGI script was invoked. For example, an HTML form might include a tag such as:

```
<INPUT TYPE = "text" NAME = "PHONE" SIZE=15>
```

which allows input of a string of length up to 15 characters into a field named `PHONE`.

After the CGI library processes the input, it provides applications with the various fields from the input form in a single Icon global variable, a table named `cgi`. The keys of this table are exactly the names given in the HTML INPUT tags. The values accessed from the keys are the string values supplied by the user. For example, to access the PHONE field from the above example, the application could write

```
cgi["PHONE"]
```

1.0.3 Processing Output

The main task of the CGI script is to generate an HTML file as its output, and for this task cgi.icn provides a host of procedures. Typically these procedures convert an Icon value into a string, wrapped with an appropriate HTML tag in order to format it properly. A typical example is the library procedure `cgiSelect(name,values)` which writes an HTML SELECT tag for a field named *name*, which generates a list of radio buttons on an HTML form whose labels are given by strings in the second parameter, a list of *values*. An Icon programmer might write

```
cgiSelect("GENDER", ["female", "male"])
```

to generate the HTML

```
<SELECT NAME="GENDER">
<OPTION SELECTED>female
<OPTION>male
</SELECT>
```

2 CGI Execution Environment

CGI scripts don't execute as standalone programs and aren't launched from a command-line, they are launched by a web server. The details of this are necessarily dependent on the operating system and web server combination in use. The following examples are based on a typical UNIX Apache server installation in which users' HTML files are installed under `$HOME/public_html`. Check with your system administrator or web server documentation for the specific names and permissions required to execute scripts from your web server.

You need to create a directory under your `public_html` directory by the name of `cgi-bin` with the appropriate permissions, namely group and other need to have read and execute permissions. **Do not give anyone but yourself write permissions!!** The following commands will complete the task above. The percent sign (`%`) is not part of the comand, it denotes the Unix shell prompt. The period in the final command is part of the command and refers to the current working directory.

```
% mkdir cgi-bin
% cd cgi-bin
% chmod go+rx .
```

Copy the ‘`cgi.icn` <http://cvs.sourceforge.net/cgi-bin/viewcvs.cgi/unicon/unicon/ipl/procs/c

Create a form like this one, called ‘`simple.html` <simple.html>`__, or save a copy of this` executable.

```
<html>
```

```
<head>
<title>
A Simple Example</title>
</head>
<body>

<h1 align=center>
<b> A Simple Example</b> </h1>
<h1 align=center> <b>
Using the CGI Icon Library</b> </h1>

<form method="GET" action="/cgi-bin/simple.cgi">

<table>
<tr><td>1. Enter your name</td>
<td> <input type="text" name="name" size=25>
</td></tr> <br>

<tr><td>2. Enter your Age:</td>
<td> <input type="text" name="age" size=2>  Years Old</td>
</tr>

</table> <table>

<tr><td>3. Favorite Food</td>
<td align="left"><input type="checkbox" Name="pizza">Pizza</td>
<td align="left"><input type="checkbox" Name="burger">Hamburger</td>
<td align="left"><input type="checkbox" Name="taco">Tacos</td>
</tr>

<tr><td>4. Favorite Color:</td>
<td align="left"><input type="checkbox" Name="red">Red</td>
<td align="left"><input type="checkbox" Name="green">Green</td>
<td align="left"><input type="checkbox" Name="blue">Blue</td>
</tr>

<tr><td>5. Education: </td>
<td align="left"><input type="checkbox" Name="bs">BS</td>
<td align="left"><input type="checkbox" Name="ms">MS</td>
<td align="left"><input type="checkbox" Name="phd">PHD</td>
```

```
</tr>

</table><br><br><br>
Comments:<br> <textarea rows = 5 cols=60 name="comments"></textarea>
<br> <br><br>
<input type="submit" value="Submit Data">
<input type="reset" value="Reset Form">
</form>

</body>
</html>
```

To produce the "simple.cgi" script that is invoked from the FORM tag above, save 'simple.icn <simple.icn>__ as text (by right clicking your mouse on t (Many web servers are configured so that CGI script executables must end with the extension .cgi , Check with your System Administrator if the .cgi extension does not work for you.)

```
link cgi      # This links the PRECOMPILED cgi library and makes
              # those functions availible to you

procedure cgimain()  # This is the start of the script

host := getenv("REMOTE_ADDR")  # This is an example of using the
                               # Environment Variables to get some information

write("Submitted IP address:  ", host,"<br>")

write("Submitted on: ", &dateline)       #&dateline is an Icon feature
```

```
# Example reading form input specified in simple.html line 15
write("<br>Hello, ", cgi["name"]," welcome to the output page.<br>")

write("Your age is : ", cgi["age"],"<br>")

write("And I see you enjoy:")
write(if cgi["pizza"] === "on" then "Pizza" else "", " ",
```

```
         if cgi["burger"] === "on" then "Hamburgers" else "", " ",
         if cgi["taco"] === "on" then "Tacos" else "", " ")
write(" for your every meal.<br>")

write("And your favorite color to look at is")
write(if cgi["red"] === "on" then "Red<br>" else "", " ",
         if cgi["green"] === "on" then "Green<br>" else "", " ",
         if cgi["blue"] === "on" then "Blue<br>" else "", " ")

write("Education: ")
write(if cgi["bs"] === "on" then "BS<br>" else "", " ",
         if cgi["ms"] === "on" then "MS<br>" else "", " ",
         if cgi["phd"] === "on" then "PHD<br>" else "", " ")
write("<br><br>")

write("Your words of wisdom are:<br><br>  ",cgi["comments"],"<br>")

write("<br><br><center>If you changed your mind... ")
write("<A href=   "simple.html   "> You can always Go Back!</a>")
write("  fix it  and re submit it!</center><br><br>")

end
```

3 Another Simple Example

Verification Screen

A lot happens in the following example script ‘`simple2.icn` <simple2.icn>`__ . Use the comm` the only change needed is to change the line:

`<form method="GET" action="/cgi-bin/simple.cgi">`

to

`<form method="GET" action="/cgi-bin/simple2.cgi">`

within the `simple.html` document

The comments will help explain what is happening.

```
link cgi          # This links the PRECOMPILED cgi library and makes
                  # those functions available to you
```

```
procedure cgimain()  # This is the start of the script

host := getenv("REMOTE_HOST")  # This is an example of using the
                               # Environment Variables to get some information

# The following line opens a file pointed to by f
# The cgiEcho is used to  output the contents
# of the subsequent write()'s to both the file pointed to by  f and
# the screen. (with HTML tags for newlines)  If for some reason the
# file can not be opened, the script would stop.

f := open("/tmp/temp.file", "w") | stop("Can't open file ")

cgiEcho(f, "Submitted IP address:  ", host,"")
cgiEcho(f, "Submitted on: ", &dateline) #This is an Icon feature
cgiEcho(f, "Hello, ", cgi["name"]," welcome to the output page.")
cgiEcho(f, "Your age is : ", cgi["age"],"")

cgiEcho(f, "And I see you enjoy:", if cgi["pizza"] === "on" then "Pizza" else "", " ",
        if cgi["burger"] === "on" then "Hamburgers" else "", " ",
        if cgi["taco"] === "on" then "Tacos" else "", " "," for your every meal.")

cgiEcho(f,"And your favorite color is ",if cgi["red"] === "on" then "Red" else "", " ",
        if cgi["green"] === "on" then "Green" else "", " ",
        if cgi["blue"] === "on" then "Blue" else "", " ")

cgiEcho(f,"Education: ",if cgi["bs"] === "on" then "BS" else "", " ",
        if cgi["ms"] === "on" then "MS" else "", " ",
        if cgi["phd"] === "on" then "PHD" else "", " ")

cgiEcho(f,"")
cgiEcho(f,"Your words of wisdom are:   ",cgi["comments"],"")

write("<center>If you changed your mind... ")
write("<a href=./simple.html> You can always Go Back !</a>")
write("  fix it  and re submit it!</center>")

# If you had opened the file and used the cgiEcho() procedure, then you
# could use this system() call to that file mailed to you. And of course
# clean up after yourself.
```

```
        system("cd /tmp; mail jvallejo < temp.file; rm temp.file")
end
```

You will have to experiment with the above code and the cgi library functions to get your scripts working correctly. You might want to get these examples working first and then adapt them to your needs.

4 CGI Library Reference

4.0.1 cgi – table containing input fields

Library global variable cgi is a table whose keys are the names of input fields in the invoking HTML page's form, and whose values are whatever the user typed in those input fields.

4.0.2 cgiInput(type, name, values) – generate INPUT tags

Generates HTML INPUT tags of a particular `type` with a certain `name` for each element of a list of `values`. The first value's input tag is CHECKED.

4.0.3 cgiSelect(name, values) – generate SELECT tags

Generates an HTML SELECT tag with a certain `name` and embedded OPTION tags for each element of a list of `values`. The first value's OPTION tag is SELECTED.

4.0.4 cgiXYCoord(hlst) : s – convert XY coordinates to corresponding list element

This procedure is used with a ISMAP to figure out what the x and y coords and if they are between a certain boundary. It returns the value of the list that was entered.

4.0.5 cgiMyURL() : s – return the URL for the currently executing CGI script

Return the URL for the current script, as obtained from the SERVER_NAME and SCRIPT_NAME environment variables.

4.0.6 cgiMethGet() : – succeed if the HTTP REQUEST_METHOD is a GET, otherwise fail

Determine whether the script was invoked via GET. Called automatically during script initialization prior to `cgimain()`.

4.0.7 cgiReadParse() : T – read CGI input fields

This procedure gets input from either QUERY_STRING or stdin puts the values with their field names and returns a table that maps input field names (the keys/indices of the table) to their values. Called automatically during script initialization prior to `cgimain()`.

4.0.8 cgiPrintVariables(T) – generate HTML for an Icon table

Prints the keys and values in a table, uses simple HTML formatting.

4.0.9 cgiError(L) – generate HTML error message

Generates an error message consisting of the strings in list L, with L[1] as the title and subsequent list elements as paragraphs.

4.0.10 cgiHexVal(c) – return integer value for hexadecimal character c

Produces a value from 0 to 15 corresponding to a hex char from 0 to F.

4.0.11 cgiHexChar(c1,c2) – return char corresponding to hex digits

Produces an 8-bit char value corresponding to two hex digits encoded as chars.

4.0.12 cgiColorToHex(s) : s – return 24-bit hex color values correponding to color name

Produces a 24-bit hex color value corresponding to a string color name. At present, only the colors black, gray, white, pink, violet, brown, red, orange, yellow, green, cyan, blue, purple, and magenta are supported.

4.0.13 cgiPrePro(filename, def) – return 24-bit hex color values correponding to color name

This procedure selectively copies out parts of a named (HTML) file, writing out either anything between ALL and the value that are passed into the procedure.

4.0.14 cgiRndImg(L, s) – write a random embedded HTML image from list L

An HTML IMG tag is generated for a random element of L, which should be a list of image filenames. The tag has ALT text given in string s.

4.0.15 cgiOptwindow(opts, args...) – open a window on server or client's X display

An attempt is made to open an Icon window, either on the X server or else on display :0 of the client's machine (as defined by the IP address in REMOTE_ADDR). The Icon window is typically used to generate a .GIF image to which a link is embedded in the CGI's output.

4.0.16 main(args...) – initialize CGI script application

The CGI `main()` procedure generates an HTML header, parses the CGI input fields into a global table `cgi`, generates a background by calling the user's `cgiBBuilder()` function, if any, and calls the user's `cgimain()` function.

4.0.17 cgiBBuilder(args...) : T – generate CGI page background

If the user application defines this function, it should return a table which contains keys "background", "bgcolor", "text", "link", "vlink", "bgproperties" with appropriate values to go into the BODY tag and define background color and texture for the CGI page.

4.0.18 cgimain(args...) – execute user CGI application code

The CGI application must define procedure `cgimain()`, which generates the HTML content body for the client's web page as a result of their invoking the CGI script.

5 PHP

PHP stands for Perl Hypertext Processor. It is a popular way of introducing dynamic server-side behavior into a webpage, easier than using CGI or a servlet engine. PHP programs mostly look like HTML documents with some dynamic elements. A complete description of PHP is beyond the scope of this report, see www.php.net_.

There are two ways that PHP and Unicon can interoperate: a PHP program can invoke an external program (such as an Icon or Unicon program), or a Unicon program can write out PHP and invoke the php translator as a postprocessor. Invoking Unicon from PHP looks like

```
passthru("/your-path/your-unicon-program");
```

This is a PHP statement. While the Unicon program should write out HTML, it is not a CGI and should not expect a CGI environment. In order to use PHP from within a Unicon CGI script, you will need to have a PHP implementation which supports a PHP standalone executable, not just a PHP that runs built-in to your web server. For example, your machine might have a /usr/bin/php. In that case, using PHP in your CGI is simple, just open a pipe for writing with open("/usr/bin/php", "pw"), and write your output to the pipe.

| **Well, it is that easy on some web servers, and not that easy on** others. Some command-line arguments to /usr/bin/php might be needed. You may also have to study the web server configuration in order to make sure all the permissions are setup correctly.

6 Appendix A: Example CGI application

```
############################################################################
#
#       File:     appform.icn
#
#       Subject:  Program: CGI script that processes scholarship applications
#
```

```
#       Authors:  Clinton Jeffery
#
#       Date:     July 11, 1997
#
############################################################################
#
# This program processes a bunch of input fields defined in an on-line
# scholarship application at scholarform.html (Note: this link is broken at present)
# and from them, generates a latex file which it typesets, prints, and e-mails
# to the scholarship coordinator.
#
############################################################################

link cgi

procedure cgimain()
  every k := key(cgi) do {
      s := ""
      cgi[k] ? {
          while s ||:= tab(upto('#%$&_{}"        ^~|<>-')) do {
              case c := move(1) of {
                  "           ": s ||:= "$           setminus$"
                  "^": s ||:= "$^{                  wedge}$"
                  "~": s ||:= "$                     sim$"
                  "|": s ||:= "$                     mid$"
                  "<": s ||:= "$<$"
                  ">": s ||:= "$>$"
                  "-": s ||:= "$-$"
                  default: {
                      s ||:= "                  "; s ||:= c
                  }
              }
          }
          s ||:= tab(0)
          }
      cgi[k] := s
  }
  f := open("/tmp/appform.tex", "w") | stop("can't open /tmp/appform.tex")

  write("Generating typeset copy of application form...")
```

```
write(f,"                   documentstyle[11pt]{letter}")
write(f,"                          pagestyle{empty}")
write(f,"")
write(f,"           setlength{           textwidth}{6.6in}")
write(f,"           setlength{           textheight}{10in}")
write(f,"           setlength{           topmargin}{-.3in}")
write(f,"             setlength{             headsep}{0in}")
write(f,"          setlength{          oddsidemargin}{0in}")
write(f,"                           baselineskip 13pt")
write(f,"                            begin{document}")
write(f,"")
write(f,"                              begin{center}")
write(f,"{                     large                    bf")
if (/ (cgi["TESTSCORES"])) | trim(string(cgi["TESTSCORES"])) === "0" then
   write(f,"NSF Computer Science Scholars and Mentors Program Application")
else
   write(f,"NSF Summer Computer Science Institute Application")
write(f,"}")
write(f,"                                end{center}")
write(f,"")
write(f,"                         begin{tabular}{llll}")
writes(f, "Name: & ", cgi["NAME"])
writes(f," & Phone: & ", cgi["PHONE"])
write(f,"                                                 ")
write(f,"Address: & ", cgi["ADDRESS1"], " & Social Sec. Number: & ", cgi["SOC"], "")
write(f," & ", cgi["ADDRESS2"], " & Gender (M/F): & ", cgi["GENDER"])
write(f,"                                end{tabular}")
write(f,"")
write(f,"Semester hours completed: Overall ", cgi["TOTALCREDITS"], " in Computer Scien
write(f,"College GPA:  Overall ", cgi["COLLEGEGPA"],
      " in Computer Science courses ", cgi["CSGPA"],"         ")
if (/ (cgi["TESTSCORES"])) | trim(string(cgi["TESTSCORES"])) === "0" then
    write(f,"Are you interested in graduate studies?          ")
else
    write(f,"Are you interested in a CS degree at UTSA?")
write(f, if cgi["YES"] === "on" then "yes" else "", " ",
      if cgi["NO"] === "on" then "no" else "", " ",
      if cgi["MAYBE"] === "on" then "maybe" else "",
      "                                                  ")
```

```
write(f,"Present Employer: ", cgi["EMPLOYER"], "            ")
write(f,"Position: ", cgi["POSITION"], "Hours/week: ", cgi["HOURS"], "")
write(f,"If selected for the NSF CS Scholars program, will the scholarship enable you"
write(f,"to quit your present job?  If not, how many hours will you be working?")
write(f, cgi["STILLWORKING"], "                              ")
write(f,"Educational Background                            ")
write(f,"High School: List name, dates attended, GPA, graduated?")
write(f, cgi["HIGH1"], "                                     ")
write(f, cgi["HIGH2"], "                                ")
if (/ (cgi["TESTSCORES"])) | trim(string(cgi["TESTSCORES"])) === "0" then
    write(f,"                                          ")
else
    write(f,"Test Scores: ", cgi["TESTSCORES"], "            ")
write(f,"For each college, list name, dates attended, hours completed, degrees awarded
write(f,cgi["COLL1"], "                                      ")
write(f,cgi["COLL2"], "                                      ")
write(f,"                                                   ")
write(f,"                                                   ")
write(f,"Academic honors, scholarships, fellowships, and assistantships")
write(f,cgi["HONOR1"], "                                     ")
write(f,cgi["HONOR2"], "                                     ")
write(f,"                                                   ")
write(f,"Extracurricular interests:     hrulefill           ")
write(f,cgi["EXTRA1"], "                                     ")
write(f,cgi["EXTRA2"], "                                     ")
write(f,"Memberships in professional organizations: hrulefill  ")
write(f,cgi["ORGS1"], "                                      ")
write(f,cgi["ORGS2"], "                                      ")
write(f,"Research interests and publications, if any: hrulefill ")
write(f,cgi["RESEARCH1"], "                                   ")
write(f,cgi["RESEARCH2"], "                                   ")
write(f,"Military Service or Draft Status:    hrulefill       ")
write(f,cgi["MIL1"], "                                       ")
write(f,cgi["MIL2"], "                                       ")
write(f,"Name(s) of at least one person you have asked to complete a confidential")
write(f,"academic reference letter.                          ")
write(f,"Name    hfill Address    hfill Relationship         ")
write(f,cgi["REF1"], " ", cgi["REFADD1"], " ", cgi["REFREL1"],"")
write(f,cgi["REF2"], " ", cgi["REFADD2"], " ", cgi["REFREL2"],"")
write(f,"")
```

```
  write(f,"On the back of this sheet or an attached letter, please include a {em")
  write(f,"short/} statement of purpose, including information about your background,")
  write(f,"major and career interests, and professional goals.")
  write(f,"")
  write(f,"I certify that information provided on this application and supporting")
  write(f,"documents is correct and complete to the best of my knowledge. ")
  write(f,"")
  write(f," noindent Signature: rule{3.5in}{.01in} Date: hrulefill")
  write(f,"")
  write(f, "                                pagebreak")
  write(f,"")
  write(f, cgi["INFO"])
  write(f,"                              end{document}")
  close(f)
  write("Mailing form to program director...")
  system("cd /tmp; mail jeffery <appform.tex")
  write("Typesetting and Printing hard copy...")
  system("cd /tmp; /usr/local/bin/latex appform >/dev/null 2>/dev/null; /usr/local/bin/d
  write("Thank you for applying, ", cgi["NAME"], ".  Your application has been submitted
end
```

7 Appendix B: the `cgi.icn` library source

```
############################################################################
#
#       File:     cgi.icn
#
#       Subject:  Procedures for writing CGI scripts
#
#       Authors:  Joe Van Meter and Clinton Jeffery
#
#       Date:     August 17, 1996
#
############################################################################
#
# This library makes programming cgi programs easier by automatically
```

```
# checking for title and body procedures.  There are other procedures
# that do some repetitive things for the programmer.
#
############################################################################

global cgi                      # table of input fields

#
# cgiEcho(file,args[]) - write a file to both HTML stdout and a regular
#  text file, if one is present
#
procedure cgiEcho(f, args[])
   push(args, f)
   if type(f) == "file" then {  # if we have a file
      write ! args              # write to it
      pop(args)                 # and then discard it
      }
   put(args, "<br>")            # write HTML
   write ! args
end

#
# cgiInput(type, name, values) -
#
procedure cgiInput(ty,nam,va)
   every is := !va do {
      writes("[<INPUT TYPE="",ty,"" NAME="",nam,"" VALUE="",is,"""")
      if is===va[1] then
          writes(" CHECKED")
      write(">", is, "]")
      }
end

#
# cgiSelect(name, values)
# this program with the name and value makes a select box
#
procedure cgiSelect(nam, va)
   write("<SELECT NAME=          "", nam, "          ">")
   every is := !va do {
```

```
      writes("<OPTION" )
      if is==va[1] then writes(" SELECTED")
      write(">", is)
      }
   write("</SELECT>")
end

#
# cgiXYCoord()
# This procedure is used with a ISMAP to figure out what the x and y coords
# and if they are between a certain boundary. It returns the value of the
# list that was entered.
#
record HMap(value,x1,y1,x2,y2)

procedure cgiXYCoord(hlst)
   title := hlst[1]
   getenv("QUERY_STRING") ? {
      x := tab(find(","))
      move(1)
      y := tab(0)
      }
   every q := 2 to *hlst do {
      if (hlst[q].x1 < x < hlst[q].x2) & (hlst[q].y1 < y < hlst[q].y2) then {
         title := hlst[q].value
         }
      }
   return title
end

procedure cgiMyURL()
   return "http://" || getenv("SERVER_NAME") || getenv("SCRIPT_NAME")
end

procedure cgiMethGet()
   if (getenv("REQUEST_METHOD")==="GET") then return
   # else fail
end

#
```

```
# cgiReadParse()
# This procedure gets input from either QUERY_STRING or stdin puts the
# values with their variable names and returns a table with references
# from the variables to their values
#

procedure cgiReadParse()
static hexen
initial {
   hexen := &digits ++ 'ABCDEF'
   }
   html := [ ]
   it := ""
   cgi := table(0)
   if cgiMethGet() then
      line := getenv("QUERY_STRING")
   else line := reads(&input, getenv("CONTENT_LENGTH"))

   line ? {
      while put(html, tab(find("&"))) do {
         tab(many('&'))
         }
      put(html, tab(0))
      }
   every r := 1 to *html do
      html[r] := map(html[r], "+", " ")
   every !html ? {
      # does this really loop multiple times?  If so, what are we
      # throwing away?
      while k := tab(find("=")) do
         tab(many('='))
      data := tab(0)

      while data ?:= ((tab(find("%")) ||
                     (move(1) &
                      (c1 := tab(any(hexen))) & (c2 := tab(any(hexen))) &
                      cgiHexchar(c1,c2)) ||
                       tab(0)))

      cgi[k] := data
```

```
      }
   return cgi
end

#
# procedure cgiPrintVariables
# prints the variables with their value
#

procedure cgiPrintVariables(in)
   write("<br>")
   every X := key(in) do
      write("<b>",X,"</b> is <i>",in[X],"</i><p>")
end

procedure cgiError(in)
   if /in then
      write("Error: Script ", cgiMyURL(), " encountered fatal error")
   else {
      write("Content-type: text/html          n          n")
      write("<html><head><title>",in[1],"</title></head>    n")
      every i := 2 to *in do
         write("<p>", in[i], "</p>")
      write("</body></html>                    n")
      }

end

procedure cgiHexval(c)
    if any(&digits, c) then return integer(c)
    if any('ABCDEF', c) then return ord(c) - ord("A") + 10
end

procedure cgiHexchar(c1,c2)
    return char(cgiHexval(c1) * 16 + cgiHexval(c2))
end

#
# procedure cgiColorToHex
# if a basic color is entered into the procedure the hex values
```

```
# is returned
#

procedure cgiColorToHex(s)
static ColorTbl
initial {
   ColorTbl:=table(0)
   ColorTbl["black"] := "000000"
   ColorTbl["gray"]  := "666666"
   ColorTbl["white"] := "ffffff"
   ColorTbl["pink"]  := "ff0099"
   ColorTbl["violet"]:= "ffccff"
   ColorTbl["brown"] := "996633"
   ColorTbl["red"]   := "ff0000"
   ColorTbl["orange"]:= "ff9900"
   ColorTbl["yellow"]:= "ffff33"
   ColorTbl["green"] := "339933"
   ColorTbl["cyan"]  := "ff66cc"
   ColorTbl["blue"]  := "0000ff"
   ColorTbl["purple"]:= "990099"
   ColorTbl["magenta"]:="cc0066"
   }

   if rv := ColorValue(s) then {
      # unfinished; convert 16-bit decimal values into 8-bits/component hex
      }
   return ColorTbl[s]
end

#
# Procedure cgiPrePro
# This procedure goes through a file writes out
# either anything between ALL and the value that are passed into the
# procedure.
#

procedure cgiPrePro(filename,def)
   AllFlag := 0
   DefFlag := 0
   all := "<!-- ALL"
```

```
   look := "<!-- "||def
   intext := open(filename)
   while here:=read(intext) do {
      if match(all,here) then {
         if AllFlag = 1 then
             AllFlag := 0
         else {
            here := read(intext)
            AllFlag := 1
            }
         }
      if match(look,here) then
         if DefFlag = 1 then {
            DefFlag := 0
            }
         else {
            DefFlag := 1
            here := read(intext)
            }
      if AllFlag = 1 then writes(here)
      else if DefFlag = 1 then writes(here)
      }
end

#
# Procedure cgiRndImg
#
# if a list is passed into the procedure then an img is randomized
#

procedure cgiRndImg(GifList,AltText)
   writes("<img src= "",?GifList," "", " alt= "",AltText," "", ">")
end

procedure cgiOptwindow(opts, args[])
   if not getenv("DISPLAY") then {
      /opts["D"] := getenv("REMOTE_ADDR") || ":0"
      }
   return optwindow ! push(args, opts)
end
```

```
#
# procedure main
#
# This procedure checks for a title procedure and a body procedure and places
# the html headers and tail... it then calls the users cgimain.
#

procedure main(args)
   write("Content-type: text/html          n          n")
   write("<html>")
   if                        cgititle then {
      write("<title>")
      write(cgititle(args))
      write("</title>")
      }
   writes("<body")
   if                       cgiBBuilder then {
      BB := cgiBBuilder(args)
      writes(" background=      "",BB["background"],"      "")
      writes(" bgcolor=        "",BB["bgcolor"],"        "")
      writes(" text=          "",BB["text"],"          "")
      writes(" link=          "",BB["link"],"          "")
      writes(" vlink=         "",BB["vlink"],"         "")
      writes(" bgproperties=    "",BB["bgproperties"],"    "")
      }
   write(">")
   cgiReadParse()
   cgimain(args)
   write("</body>")
   write("</html>")
end
```

8 Appendix C: Common CGI Environment Variables

Much of the official CGI definition_ consists of a description of a set of standard environment variables that are set by the web server as a method

of passing information to the CGI script. Icon programmers access these environment variables using `getenv()`, as in

```
getenv("QUERY_STRING")
```

The following abbreviated summary of the CGI environment variables is provided as a convenient reference for Icon programmers so that this document can serve as a stand-alone reference for writing CGI scripts. For a complete listing of all of the Environment Variables supported by CGI go to http://hoohoo.ncsa.uiuc.edu/cgi/env.html

Variable	Explanation
CONTENT_LENGTH	The length of the ASCII string provided by the method="POST"
QUERY_STRING	All of the information which follows the ? in the URL when using met
REMOTE_ADDR	Contains the IP address of the client machine.
REMOTE_HOST	Contains the hostname of the client machine. Defaults to IP held by R
REQUEST_METHOD	Contains "GET" or "POST" depending on which method was used.
SERVER_NAME	The server's hostname, defaults to IP address.